

Rapport de Projet: WorldCup 2026

Ticketing API

Licence MIASHS L-2

Membre du groupe :

HAMIDI Mariam

PERRIN Clément

Sommaire :

- Présentation du Projet
 - Répartitions des tâches
 - Comment le lancer
- Structuration du Projet
 - Domain
 - Application
 - Infrastructure
- Liste des Endpoints de l'API
- Etat du projet
 - Fonctionnalités réalisées
 - Problèmes rencontrés et solutions
 - Dysfonctionnements éventuels

1. Présentation du Projet

Ce projet consiste en la création d'une API de billetterie pour la Coupe du Monde 2026. L'objectif est de permettre aux utilisateurs de consulter les matchs par ville, stade ou équipe, et d'acheter des billets en choisissant un siège spécifique.

Chacun des membres du groupe a réalisé le projet dans son entièreté (les 6 TDs), ce qui nous a permis de vite se mettre d'accord sur le rendu final et la robustesse du code. Les tâches demandées ont donc été réalisées entièrement et séparément par Hamidi Mariam et Perrin Clément, sans répartitions de celles-ci. On s'est tout de même consultés pendant toute la réalisation des tds, afin de garder une cohérence entre les deux « projets », pour pouvoir les combiner/associé plus facilement à la fin, l'utilisation de github a également facilité ce point.

Le code a été fait en respectant la séparation des responsabilités en utilisant une Architecture Hexagonale, ce qui permet de séparer la logique métier (le Domaine) de la base de données (TypeORM) ou le framework web (Hono).

Le projet est entièrement conteneurisé grâce à Docker. L'utilisation de conteneurs permet d'isoler la base de données (et l'application) du système hôte.

Comment lancer le projet

1. Lancement de l'environnement (Base de données et conteneurs) : docker compose up
2. Installation des dépendances : bun install
3. Initialisation de la base de données (Seeding) : bun run seed
4. Lancement du serveur en mode développement : bun run dev
5. Tests : Ouvrez Bruno dans l'environnement dev (localhost:3000), importez la collection de tests, et lancez les requêtes vers l'API.

2. Structuration du Projet

Le projet suit une structure stricte divisée en trois couches :

A. Domain (Le Cœur Métier)

Contient les règles de gestion et les entités pures, sans aucune dépendance technique.

- Entities : Match, Team, Stadium, City, Country, Ticket, Customer.
- Enums : MatchStatus, MatchStage.
- Errors : Classes d'erreurs (NotFoundError, ValidationError, ConflictError).

B. Application (Les Services)

Contient la logique de l'application et orchestre les entités.

- Services : MatchService, CityService, CountryService, StadiumService, TeamService, TicketService.
- Chaque service utilise des Repositories injectés pour accéder aux données.

C. Infrastructure (Les Détails Techniques)

Contient les implémentations liées aux outils externes.

- Database : Configuration de TypeORM (AppDataSource) et schémas de base de données.
- Handlers : Contrôleurs Hono (ex: GetMatchesHandler, CreateTicketHandler) qui reçoivent les requêtes HTTP et appellent les services.
- Routes : Définition des points d'entrée (endpoints) de l'API.

3. Liste des Endpoints de l'API

L'API est architecturée autour de plusieurs routeurs dédiés (via Hono) pour séparer proprement les différents domaines métiers. Voici l'ensemble des endpoints disponibles :

A. Routes de base (Home)

- GET / : Point d'entrée principal de l'API (Accueil).
- GET /health : Vérification de l'état de santé du serveur.

B. Gestion des Matches (/matches)

- GET /matches : Récupère la liste de tous les matches (avec possibilité de filtrer par date ou par équipe via les paramètres d'URL).
- GET /matches/:id : Récupère les détails d'un match spécifique grâce à son identifiant unique.
- GET /matches/:stage : Récupère un match spécifique selon sa phase (ex : final, group).
- GET /matches/stages/:stage : Récupère la liste complète des matches d'une phase précise de la compétition.
- GET /matches/status/:status : Récupère la liste des matches selon leur statut (ex : SCHEDULED).

C. Gestion des Équipes (/teams)

- GET /teams : Récupère la liste de toutes les équipes (permet le tri par ordre alphabétique croissant ou décroissant).
- GET /teams/:code : Récupère les détails d'une équipe spécifique via son code FIFA (3 lettres).

- GET /teams/:code/matches : Récupère la liste de tous les matchs joués par une équipe spécifique.
- GET /teams/:code/matches/:stage : Récupère les matchs d'une équipe pour une phase de compétition précise.

D. Gestion des Villes (/cities)

- GET /cities : Récupère la liste de toutes les villes hôtes.
- GET /cities/:name : Récupère les détails d'une ville spécifique par son nom.
- GET /cities/:name/matches : Récupère la liste des matchs qui se déroulent dans une ville spécifique.

E. Gestion des Pays (/countries)

- GET /countries : Récupère la liste de tous les pays (avec possibilité de filtrer par nom ou par code).
- GET /countries/:code : Récupère les détails d'un pays via son code.
- GET /countries/:code/cities : Récupère la liste des villes hôtes situées dans un pays spécifique.

F. Gestion des Stades (/stadiums)

- GET /stadiums : Récupère la liste de tous les stades (avec de multiples filtres optionnels : nom, ville, pays, code pays).
- GET /stadiums/:name : Récupère les détails d'un stade spécifique par son nom.
- GET /stadiums/:name/matches : Récupère la liste complète des matchs prévus dans un stade spécifique.

G. Système de Billetterie (/tickets)

- POST /tickets : Permet la création d'un nouveau ticket. Le corps de la requête (JSON) doit contenir l'ID du match, la référence du siège, ainsi que les informations du client (nom, prénom, email) selon la validation ZOD. Le endpoint vérifie la disponibilité du siège avant de valider la réservation.

4. Etat du projet

Fonctionnalités réalisées

Le projet remplit l'ensemble des fonctionnalités demandées, notamment :

Architecture et Code

- Architecture Hexagonale : Séparation stricte entre le cœur métier (Domain), la logique (Services) et le Web (Handlers Hono).

Fonctionnalités Métier

- Filtrage dynamique des matchs (par date, équipe, stade, phase, statut) et tri alphabétique des équipes par exemple.
- Réservation de tickets avec vérification stricte de l'existence du match et de la disponibilité du siège (gestion des conflits de réservation).
- Injection de dépendances manuelle (Repositories injectés dans les Services).

Base de données (TypeORM)

- Gestion des relations imbriquées : récupération automatique de la ville et du pays depuis un stade par exemple.
- Flexibilité : Recherches textuelles insensibles à la casse.

Sécurité et Validation

- Utilisation de Zod : contrôler le format de certaines requêtes JSON.
- Gestion des erreurs : centralisation des erreurs métier et traduction en codes HTTP standards (400, 404, 409, 500).

Environnement et Tests

- Conteneurisation complète de l'API et de la base de données via Docker.
- Utilisation de Bun comme environnement d'exécution principal.
- Qualité : Validation du fonctionnement de tous les endpoints avec les collections de tests sous Bruno.

Problèmes rencontrés et solutions

1. Chargement des relations : Au début, certaines imbrications (comme la ville d'un stade) apparaissaient comme undefined. Nous avons résolu ça en configurant les relations dans les appels TypeORM des Services (pour la table match par exemple).
2. Séparation Web/Métier : On a dû retirer les HTTPException dans les services. Nous avons donc utilisé des erreurs de domaine (ValidationError par exemple) que les Handlers traduisent ensuite en erreurs HTTP.
3. Filtrage complexe : La gestion des filtres dans les URL nous a poussé à adopter une logique de construction dynamique des clauses where dans TypeORM.

Dysfonctionnements éventuels

- Base de données vide : L'API ne contient pas de données par défaut au premier lancement. Il faut d'abord exécuter effectuer le seeding (seed) avant de tester.
- Sensibilité à la casse : Pour certaines recherches, l'utilisation de ILike a été implémentée pour éviter les erreurs de saisie (majuscules/minuscules).